

Salesforce Developer Bridge Program

Day 2 Task Submission Report

Submitted By: Midasala Nikshitha Ramesh [23PA1A05F2]

Project: Hospital OPD Management System

Task 1 – Apex Collections

Objective:

To understand and implement Apex Collections (**List**, **Set**, and **Map**) for efficient data storage and retrieval in Salesforce.

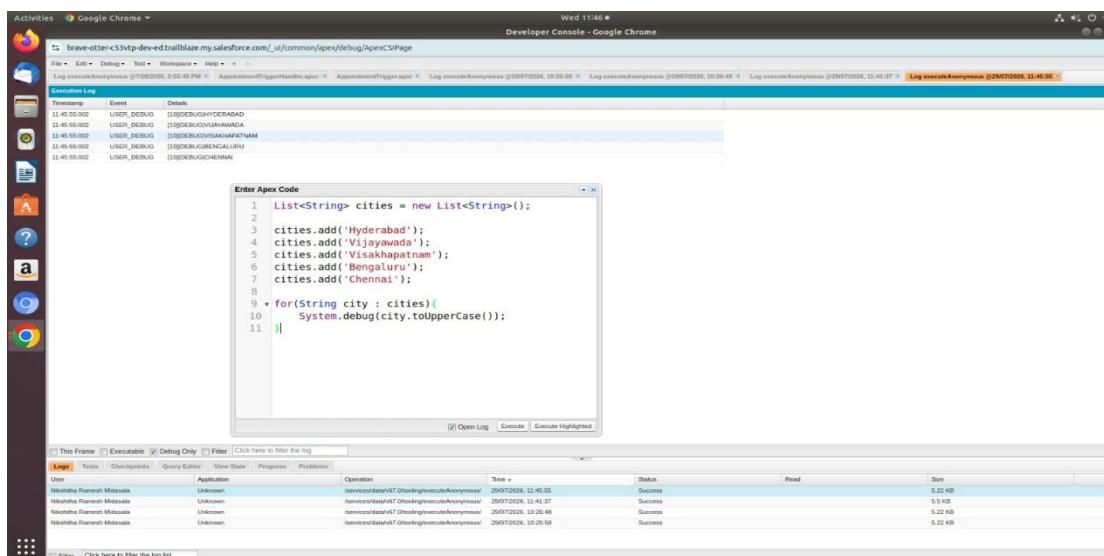
Work Performed:

- Created a **List** of city names and displayed them in uppercase.
- Used a **Set** to store unique Patient record IDs and verified the number of distinct records.
- Implemented a **Map** to retrieve patient details using a record ID.
- Executed all programs in the Developer Console and verified the output through the debug log.

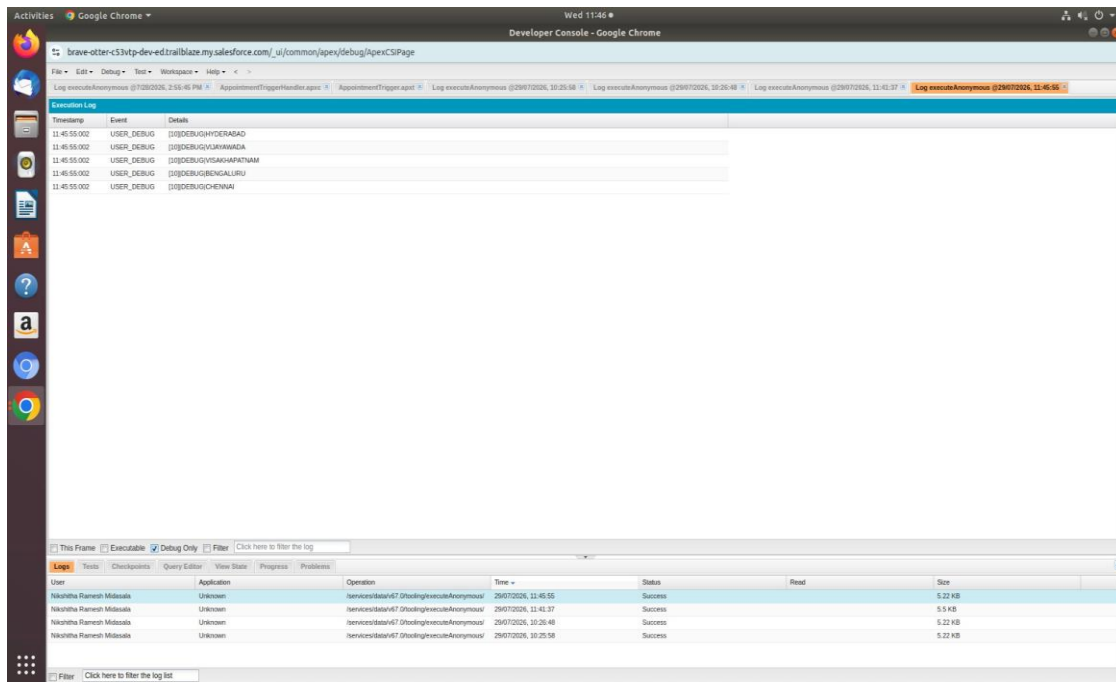
Outcome:

Successfully implemented List, Set, and Map collections in Apex and verified the expected results through the debug logs.

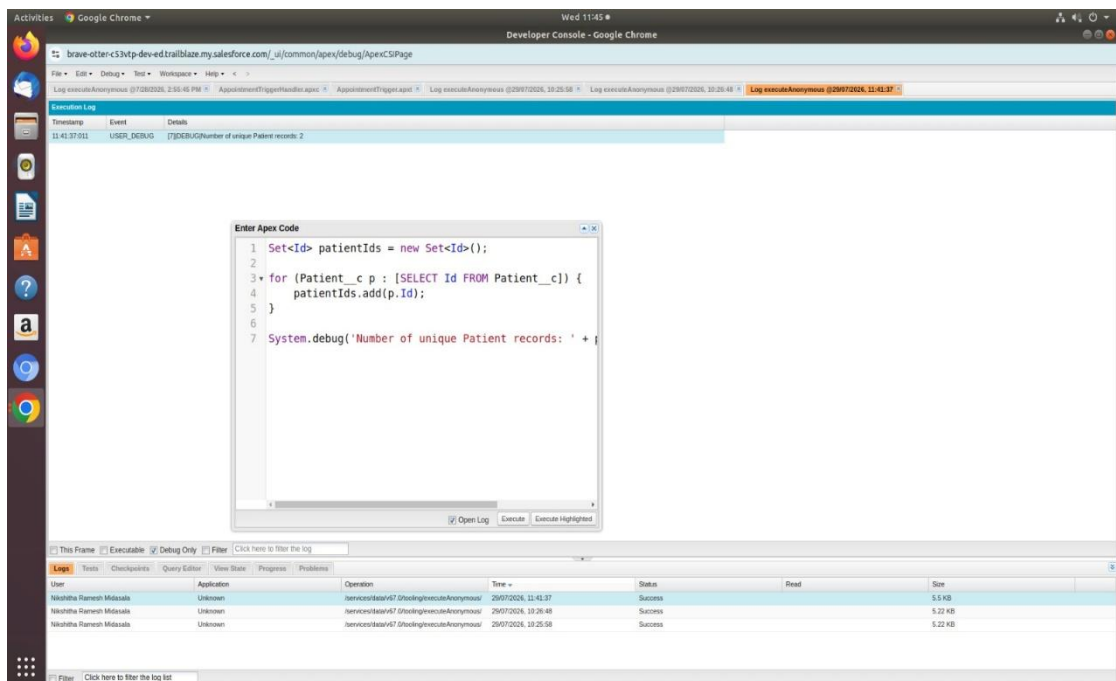
Screenshot 1.1: List Collection Code



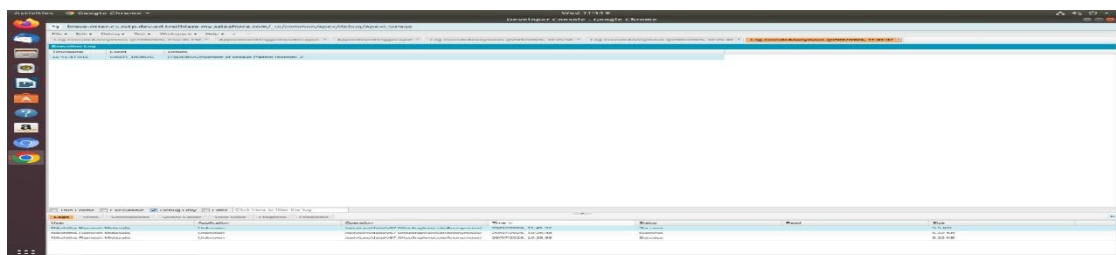
Screenshot 1.2: List Collection Output



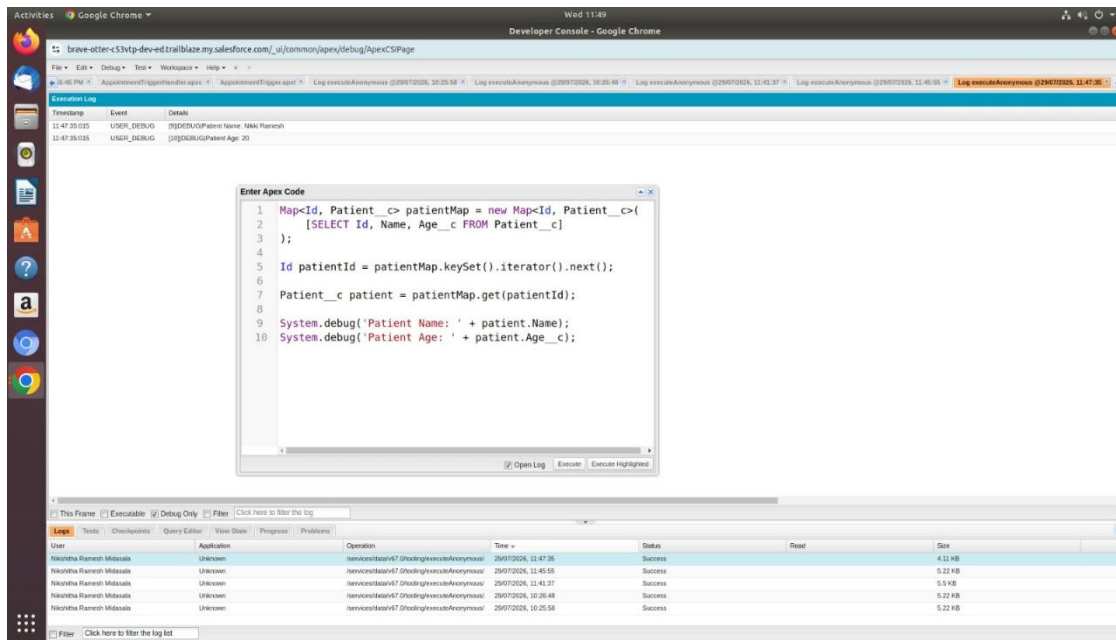
Screenshot 1.3: Set Collection Code



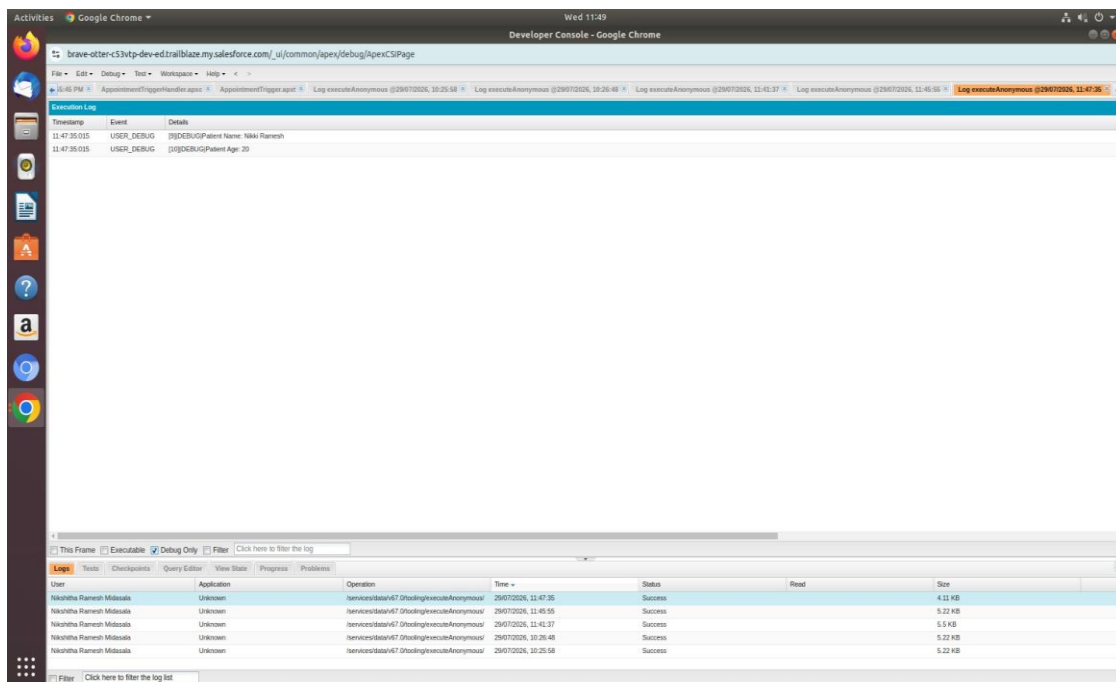
Screenshot 1.4: Set Collection Output



Screenshot 1.5: Map Collection Code



Screenshot 1.6: Map Collection Output



Task 2 – Governor Limits & Bulkification

Objective:

To understand Salesforce governor limits and optimize Apex triggers using bulkification techniques.

Work Performed:

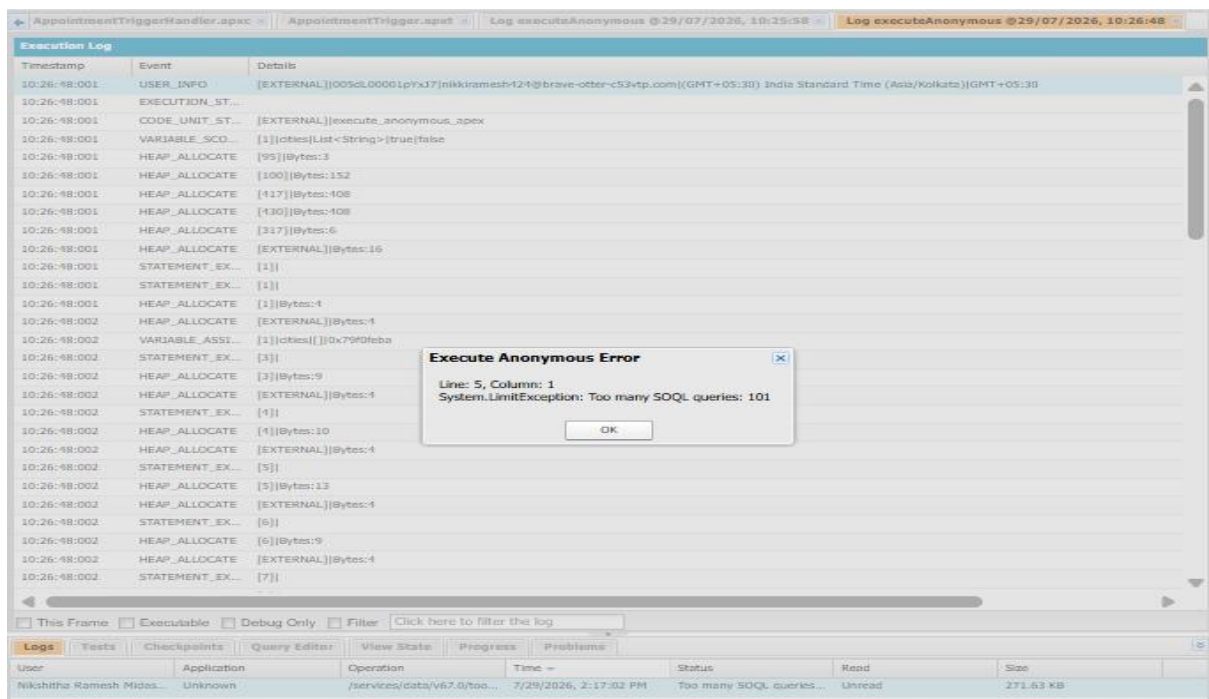
- Created a trigger with SOQL inside a loop to intentionally trigger a governor limit error.

- Identified the **"Too many SOQL queries: 101"** error during bulk record processing.
- Optimized the trigger by moving the SOQL query outside the loop.
- Tested the bulkified trigger and verified successful execution without governor limit errors.

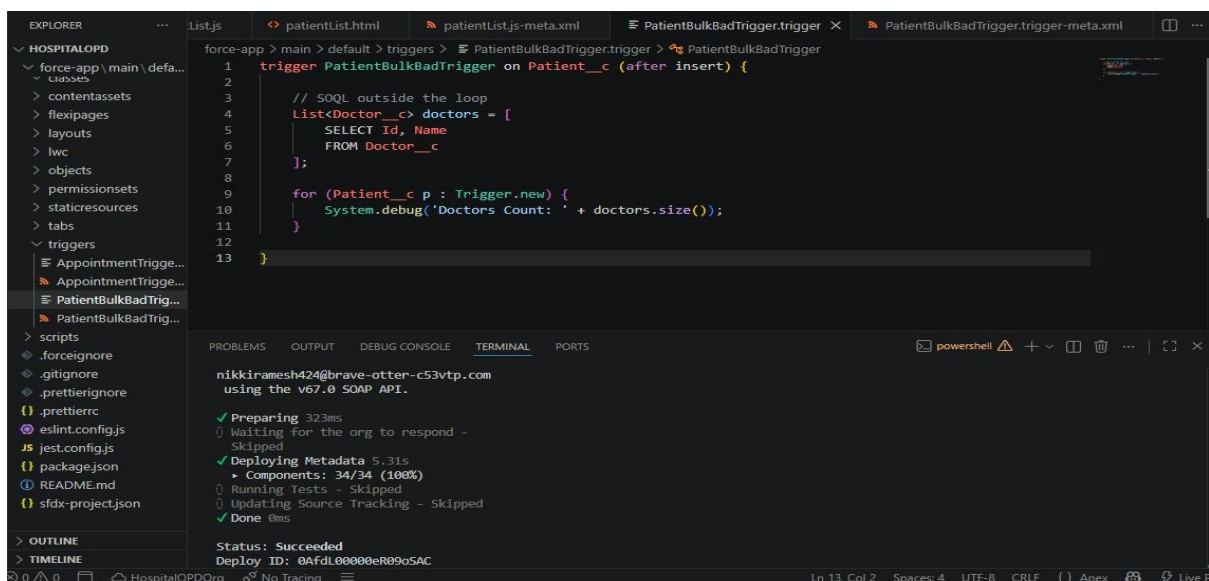
Outcome:

Successfully implemented a bulkified trigger that processed records efficiently without exceeding Salesforce governor limits.

Screenshot 2.1: Governor Limit Error



Screenshot 2.2: Bulkified Trigger Code



Screenshot 2.3: Successful Execution After Bulkification

Timestamp	Event	Details
14:22:06:523	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:523	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:523	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:523	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:523	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:523	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:524	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:525	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:525	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:525	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:525	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:525	USER_DEBUG	[10] DEBUG Doctors Count: 2
14:22:06:525	USER_DEBUG	[10] DEBUG Doctors Count: 2

User	Application	Operation	Time	Status	Read	Size
Nikshitha Ramesh Midas...	Unknown	/services/data/v67.0/too...	7/29/2026, 2:22:06 PM	Success		281.44 KB
Nikshitha Ramesh Midas...	Unknown	/services/data/v67.0/too...	7/29/2026, 2:17:02 PM	Too many SOQL queries...	Unread	271.63 KB

Task 3 – Asynchronous Apex

Objective:

To implement an asynchronous **@future** method for updating Salesforce records and understand asynchronous processing.

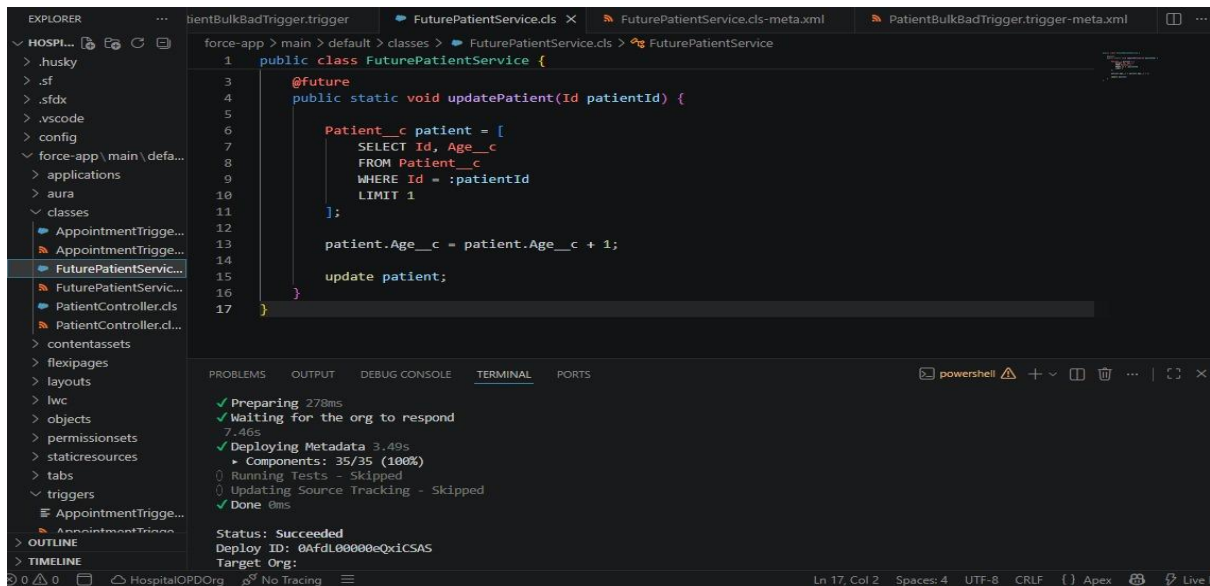
Work Performed:

- Created a **@future** method to update a Patient record asynchronously.
- Invoked the method using **Execute Anonymous** in the Developer Console.
- Verified the asynchronous execution through the **Apex Jobs** page in Salesforce Setup.

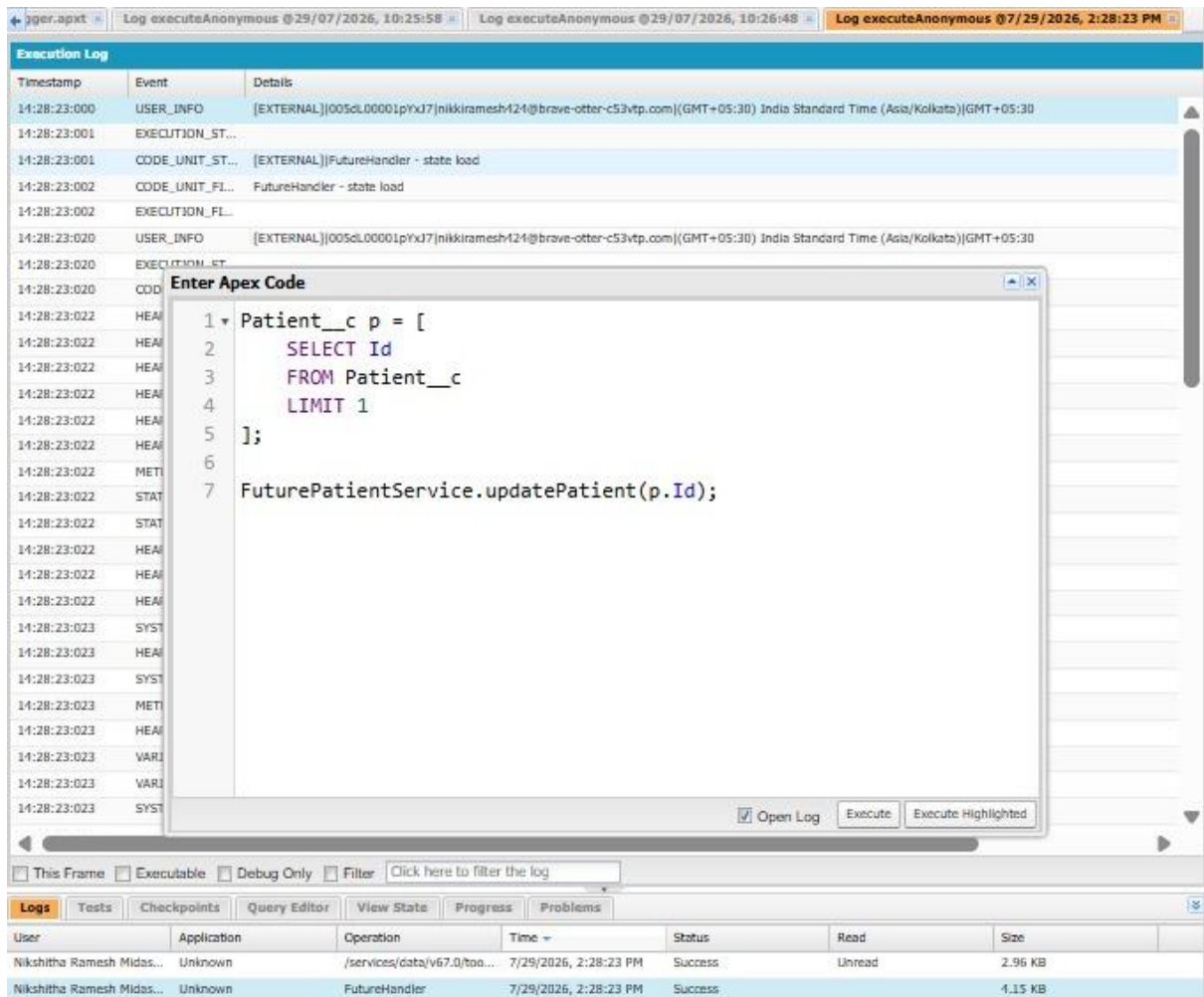
Outcome:

Successfully implemented and executed the **@future** method. The Apex Job completed successfully, confirming asynchronous record processing.

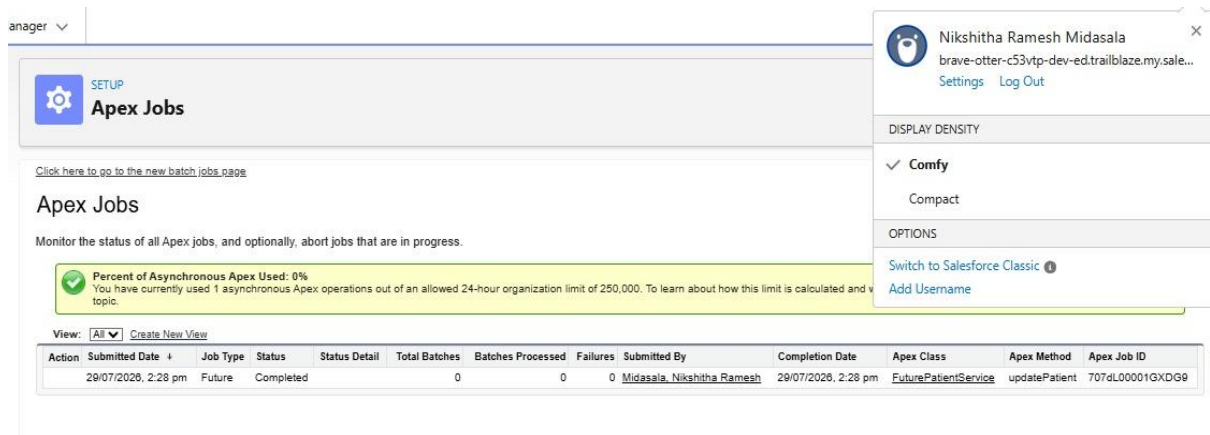
Screenshot 3.1: @future Method Code



Screenshot 3.2: Execute Anonymous Window Calling the @future Method



Screenshot 3.3: Apex Jobs Page Showing Successful Execution



Task 4 – LWC Component Communication

Objective:

To implement communication between parent and child Lightning Web Components using the `@api` property and custom events.

Work Performed:

- Created Parent and Child Lightning Web Components.
- Passed data from the parent component to the child using the `@api` property.
- Implemented child-to-parent communication using custom events.
- Added the components to the Lightning App Builder and verified their interaction on the Home Page.

Outcome:

Successfully established two-way communication between the parent and child components. The parent passed data to the child, and the child notified the parent through a custom event.

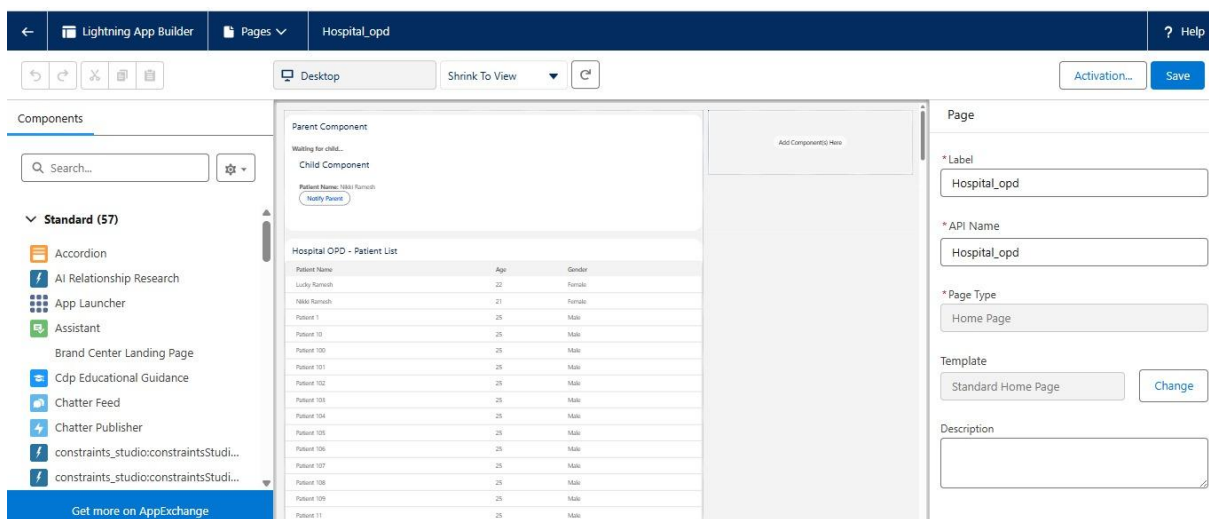
Screenshot 4.1: Child Component JavaScript (`notifyParent()` with Custom Event)



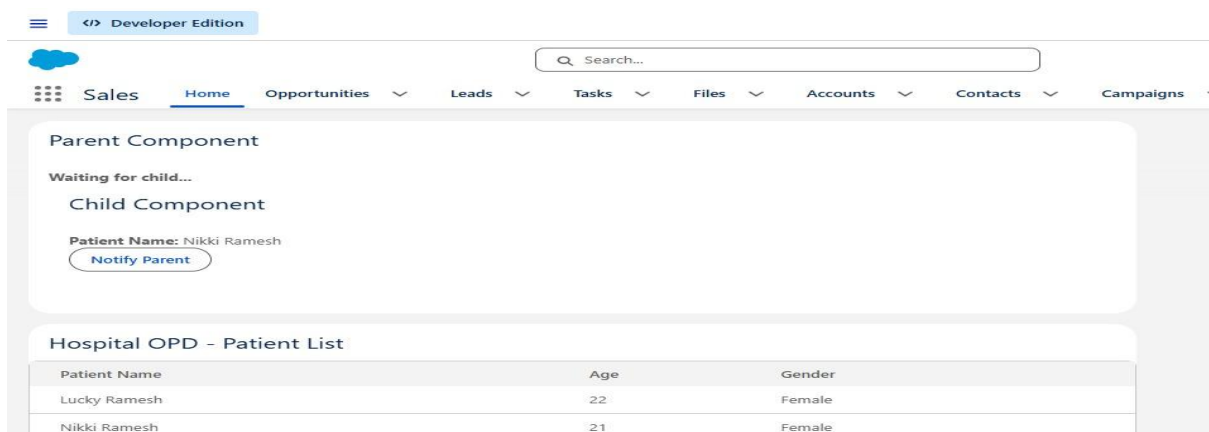
Screenshot 4.2: Parent Component JavaScript (handleButtonClick() Event Handler)

```
JS childComponent.js JS parentComponent.js X parentComponent.html parentComponent.js-m
force-app > main > default > lwc > parentComponent > JS parentComponent.js > ParentComponent
1  import { LightningElement } from 'lwc';
2
3  export default class ParentComponent extends LightningElement {
4
5      patientName = 'Nikki Ramesh';
6
7      message = 'Waiting for child...';
8
9      handleButtonClick(event) {
10         this.message = event.detail;
11     }
12
13 }
```

Screenshot 4.3: Lightning App Builder Showing Both Components



Screenshot 4.4: Parent Component Display (Waiting for Child)



Screenshot 4.5: Parent Component Updated After Child Button Click

The screenshot shows a web application interface with a top navigation bar and a main content area. The navigation bar includes a logo, a search bar, and a menu with items: Sales, Home (active), Opportunities, Leads, Tasks, Files, Accounts, Contacts, and Campaigns. The main content area is divided into two sections. The top section, titled 'Parent Component', displays a message 'Child Button Clicked!' and a 'Child Component' section. The 'Child Component' shows 'Patient Name: Nikki Ramesh' and a 'Notify Parent' button. The bottom section, titled 'Hospital OPD - Patient List', contains a table with patient information.

Patient Name	Age	Gender
Lucky Ramesh	22	Female
Nikki Ramesh	21	Female

Conclusion:

Successfully completed all Day 2 tasks and gained practical knowledge of Apex Collections, Governor Limits, Asynchronous Apex, and Lightning Web Component communication.